APC Assignment 1 – October 2026

1 Introduction

Many problems in science and engineering (e.g., the discretization of partial differential equations, least-squares fitting, circuit and structural analysis) require the solution of a linear system

$$A\mathbf{x} = \mathbf{b}, \tag{1}$$

where $A \in \mathbb{R}^{n\times n}$ and $\mathbf{b} \in \mathbb{R}^n$ are given, and $\mathbf{x} \in \mathbb{R}^n$ is the unknown. Instead of factorizing A as direct methods do, *iterative methods* build a sequence of approximations $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \ldots$ that converges to $\mathbf{x}$, and only need to compute products of A with vectors. Moreover, in most applications A is *sparse*, i.e., only a few of its n^2 entries are different from zero. Since iterative methods access A only through matrix-vector products, they can take advantage of this by storing and using only the non-zero entries. In this assignment, you will implement the most popular iterative method, the *Conjugate Gradient* (CG) method, together with its *preconditioned* version (PCG), for matrices stored in a sparse format.

Throughout the assignment, $\mathbf{u}^T\mathbf{v} = \sum_{i=0}^{n-1} u_i v_i$ denotes the scalar product of two vectors, and $\|\mathbf{v}\| = \sqrt{\mathbf{v}^T\mathbf{v}}$ the Euclidean norm. The *residual* of an approximation $\mathbf{x}_k$ is the vector $\mathbf{r}_k = \mathbf{b} - A\mathbf{x}_k$: it is zero if and only if $\mathbf{x}_k$ is the exact solution, so its norm $\|\mathbf{r}_k\|$ measures how far we are from solving (1).

1.1 Symmetric positive definite matrices

CG can only be applied when A is *symmetric positive definite* (SPD), namely

$$A = A^T \qquad \text{and} \qquad \mathbf{v}^T A\mathbf{v} > 0 \quad \text{for every } \mathbf{v} \neq \mathbf{0}.$$

Every SPD matrix is invertible, so (1) has exactly one solution. Moreover, its diagonal entries are strictly positive, since $a_{ii} = \mathbf{e}_i^T A\mathbf{e}_i > 0$, where $\mathbf{e}_i$ is the i-th vector of the canonical basis.

CG requires an SPD matrix because, in this case, solving (1) is equivalent to minimizing the quadratic function

$$\phi(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}^T\mathbf{x}. \tag{2}$$

Indeed, if A is symmetric, the gradient of ϕ is $\nabla\phi(\mathbf{x}) = A\mathbf{x} - \mathbf{b} = -\mathbf{r}$, which vanishes exactly at the solution of (1). If A is also positive definite, ϕ is a convex "bowl", and this point is its unique minimum: CG solves (1) by walking

downhill on ϕ. If A is not SPD, ϕ may have no minimum at all, and the quantity $\mathbf{p}^T A\mathbf{p}$, which appears in a denominator of the method (see below), may vanish or become negative: the algorithm then breaks down.

1.2 The Conjugate Gradient method

Starting from an initial guess $\mathbf{x}_0$, every iteration of CG moves along a *search direction* $\mathbf{p}_k$:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k.$$

The step length α_k is the one that minimizes $\phi(\mathbf{x}_k + \alpha\,\mathbf{p}_k)$ over $\alpha \in \mathbb{R}$. Setting the derivative with respect to α to zero gives

$$\alpha_k = \frac{\mathbf{p}_k^T \mathbf{r}_k}{\mathbf{p}_k^T A\mathbf{p}_k},$$

which is well defined because $\mathbf{p}_k^T A\mathbf{p}_k > 0$ for an SPD matrix. Since $\mathbf{r}_{k+1} = \mathbf{b} - A\mathbf{x}_{k+1} = \mathbf{r}_k - \alpha_k A\mathbf{p}_k$, the residual can be updated with the vector $A\mathbf{p}_k$ already computed for α_k, without any additional matrix-vector product.

The choice $\mathbf{p}_k = \mathbf{r}_k$ (steepest descent) converges slowly; CG instead chooses each new direction as a combination of the new residual and of the previous direction,

$$\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k,$$

where β_k is chosen so that $\mathbf{p}_{k+1}^T A\mathbf{p}_k = 0$ (the directions are said to be *A-conjugate*). With this choice, one can prove that all search directions are mutually A-conjugate, that all residuals are mutually orthogonal, and that the formulas simplify to

$$\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T A\mathbf{p}_k}, \qquad \beta_k = \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k}.$$

As a consequence, in exact arithmetic CG finds the exact solution in at most n iterations. In practice, CG is stopped as soon as the residual is small enough, and its speed depends on the *condition number* $\kappa(A) = \lambda_{\max}(A)/\lambda_{\min}(A)$, i.e., the ratio between the largest and the smallest eigenvalue of A. More precisely, the error $\mathbf{e}_k = \mathbf{x} - \mathbf{x}_k$ satisfies

$$\|\mathbf{e}_k\|_A \le 2\left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1}\right)^k \|\mathbf{e}_0\|_A, \qquad \text{where } \|\mathbf{v}\|_A = \sqrt{\mathbf{v}^T A\mathbf{v}},$$

so the larger $\kappa(A)$, the slower the convergence.

1.3 Preconditioning and the Jacobi preconditioner

If $\kappa(A)$ is large, CG can be accelerated by *preconditioning*. The idea is to choose an SPD matrix $M \approx A$, called *preconditioner*, such that systems with M

are very cheap to solve. Since $M^{-1}A$ is in general not symmetric, CG cannot be applied to it directly. Instead, one applies CG to the equivalent system $\left(M^{-1/2}AM^{-1/2}\right)\mathbf{y} = M^{-1/2}\mathbf{b}$, with $\mathbf{x} = M^{-1/2}\mathbf{y}$, whose matrix is SPD and has the same eigenvalues as $M^{-1}A$: if M is a good approximation of A, its condition number is much smaller than $\kappa(A)$. Neither M^{-1} nor $M^{-1/2}$ has to be computed explicitly: after rewriting the iterations in terms of $\mathbf{x}_k$, the resulting *Preconditioned Conjugate Gradient* (PCG) method is identical to CG, except that at each iteration it also computes the *preconditioned residual*

$$\mathbf{z}_k = M^{-1}\mathbf{r}_k,$$

and uses it in place of $\mathbf{r}_k$ in the scalar $\rho_k = \mathbf{r}_k^T\mathbf{z}_k$ and in the update of the search direction.

In this assignment, we use the simplest preconditioner, the *Jacobi* (or diagonal) preconditioner

$$M = \mathrm{diag}(a_{00}, a_{11}, \ldots, a_{n-1,n-1}), \quad \text{so that} \quad (\mathbf{z}_k)_i = \frac{(\mathbf{r}_k)_i}{a_{ii}}, \quad i = 0, \ldots, n-1.$$

M is SPD because $a_{ii} > 0$, and applying M^{-1} costs only n divisions. It is very effective on *badly scaled* systems, whose diagonal entries have very different magnitudes (e.g., because the unknowns are measured in different units): if $A = DTD$, with D diagonal and T well conditioned with constant diagonal, $\kappa(A)$ can be much larger than $\kappa(T)$ and CG is slow, while PCG removes the scaling D and converges as fast as CG on T. Conversely, if all the diagonal entries of A are equal, M is a multiple of the identity, and PCG computes the same iterates as CG (up to round-off errors).

Note that CG is the special case of PCG with $M = I$, that is, $\mathbf{z}_k = \mathbf{r}_k$. Therefore, a **single** algorithm covers both methods, as shown in Algorithm 1. Both variants stop when the *relative residual* satisfies $\|\mathbf{r}_k\| \le \varepsilon \|\mathbf{b}\|$, for a prescribed tolerance ε, or after a maximum number of iterations $k_{\max}$.

Without preconditioning, $\rho_k = \mathbf{r}_k^T\mathbf{r}_k$ and Algorithm 1 reduces to the CG formulas above. The stored norms are always those of $\mathbf{r}_k$ (*not* of $\mathbf{z}_k$), so that CG and PCG can be compared on the same scale. The residual history $\|\mathbf{r}_0\|, \|\mathbf{r}_1\|, \ldots, \|\mathbf{r}_k\|$ contains $k+1$ values, where k is the number of iterations actually performed.

1.4 Sparse matrices and the CSR format

The matrices used in the tests of this assignment are tridiagonal: for $n = 50$, only 148 of their 2500 entries are different from zero. Storing all the n^2 entries wastes memory, and computing $A\mathbf{x}$ with the dense formula $y_i = \sum_{j=0}^{n-1} a_{ij}x_j$ wastes time multiplying by zeros.

The *Compressed Sparse Row* (CSR) format stores only the non-zero entries of A, by means of three arrays:

- `values`: the non-zero entries, stored row by row and, within each row, from left to right;

Algorithm 1 Conjugate Gradient, with optional Jacobi preconditioning

Input: flag `use_preconditioner`

Available: SPD matrix A, right-hand side $\mathbf{b}$, initial guess $\mathbf{x}_0$, tolerance ε, max. iterations $k_{\max}$, preconditioner $M = \mathrm{diag}(A)$

Output: approximate solution $\mathbf{x}_k$, residual norms $\|\mathbf{r}_0\|, \ldots, \|\mathbf{r}_k\|$, number of iterations k

$\mathbf{r}_0 \leftarrow \mathbf{b} - A\mathbf{x}_0$ ▷ $\mathbf{x}_0$ is not necessarily zero
if `use_preconditioner` $\mathbf{z}_0 \leftarrow M^{-1}\mathbf{r}_0$ **else** $\mathbf{z}_0 \leftarrow \mathbf{r}_0$
$\mathbf{p}_0 \leftarrow \mathbf{z}_0, \quad \rho_0 \leftarrow \mathbf{r}_0^T \mathbf{z}_0, \quad k \leftarrow 0$
store $\|\mathbf{r}_0\|$
while $\|\mathbf{r}_k\| > \varepsilon \|\mathbf{b}\|$ **and** $k < k_{\max}$ **do**
    $\mathbf{q}_k \leftarrow A\mathbf{p}_k$
    $\alpha_k \leftarrow \rho_k / (\mathbf{p}_k^T \mathbf{q}_k)$
    $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \alpha_k \mathbf{p}_k$
    $\mathbf{r}_{k+1} \leftarrow \mathbf{r}_k - \alpha_k \mathbf{q}_k$
    store $\|\mathbf{r}_{k+1}\|$
    if `use_preconditioner` $\mathbf{z}_{k+1} \leftarrow M^{-1}\mathbf{r}_{k+1}$ **else** $\mathbf{z}_{k+1} \leftarrow \mathbf{r}_{k+1}$
    $\rho_{k+1} \leftarrow \mathbf{r}_{k+1}^T \mathbf{z}_{k+1}$
    $\beta_k \leftarrow \rho_{k+1} / \rho_k$
    $\mathbf{p}_{k+1} \leftarrow \mathbf{z}_{k+1} + \beta_k \mathbf{p}_k$
    $k \leftarrow k + 1$
end while

- `col_idx`: the column indices of the entries, i.e., `values[k]` is in column `col_idx[k]`;
- `row_ptr`: an array of $n+1$ elements, where `row_ptr[i]` is the position in `values` of the first entry of row i, and `row_ptr[n]` is the total number of non-zero entries. Equivalently, `row_ptr[i]` is the number of non-zero entries in the rows *before* row i, i.e., in rows $0, \ldots, i-1$ (in particular, `row_ptr[0]` $= 0$). For instance, in the example below, `row_ptr[2]` $= 5 = 2 + 3$ counts the entries of rows 0 and 1, and not those of row 2.

Therefore, the entries of row i are

$$\texttt{values}[k], \qquad k = \texttt{row_ptr}[i], \ldots, \texttt{row_ptr}[i+1] - 1,$$

and a row without non-zero entries has `row_ptr`$[i] =$ `row_ptr`$[i{+}1]$. For example, the CSR representation of the symmetric matrix

$$A = \begin{bmatrix} 4 & -1 & 0 & 0 \\ -1 & 4 & 0 & -2 \\ 0 & 0 & 5 & 0 \\ 0 & -2 & 0 & 3 \end{bmatrix} \quad \text{is} \quad \begin{aligned} \texttt{values} &= [4, -1, -1, 4, -2, 5, -2, 3] \\ \texttt{col_idx} &= [0, 1, 0, 1, 3, 2, 1, 3] \\ \texttt{row_ptr} &= [0, 2, 5, 6, 8] \end{aligned}$$

With this format, the i-th entry of the product $\mathbf{y} = A\mathbf{x}$ is computed using only

the non-zero entries of row i:

$$y_i = \sum_{k=\texttt{row_ptr}[i]}^{\texttt{row_ptr}[i+1]-1} \texttt{values}[k]\; x_{\texttt{col_idx}[k]}, \qquad i = 0, \ldots, n-1, \tag{3}$$

so that its cost is proportional to the number of non-zero entries instead of n^2.

2 Code implementation

The provided project contains the following files:

- `utils`: the `Rnvector` alias (`std::vector<double>`), the `Rnmatrix` alias (`std::vector<Rnvector>`) for dense matrices, where `A[i][j]` is a_{ij}, and the vector operations `x + y`, `x - y`, `lambda * x`, `dot(x, y)` ($= \mathbf{x}^T\mathbf{y}$) and `norm(x)` ($= \|\mathbf{x}\|$);
- `CSRMatrix`: a sparse $n \times n$ matrix in the CSR format, with the members `n`, `values`, `col_idx` and `row_ptr` described in Section 1.4, and the methods `nnz()` (number of non-zero entries), `get(i, j)` (returns a_{ij}), `diagonal()` and `is_symmetric()`;
- `JacobiPreconditioner`: its method `apply(r)` returns $\mathbf{z} = M^{-1}\mathbf{r}$;
- `CGSolver`: the solver, which stores the data of Algorithm 1 (with A in the CSR format) and its results;
- `main.cpp`: builds the test systems (with A stored as a dense `Rnmatrix`), converts A to the CSR format, runs the solver and writes the results to file.

After carefully reading the provided files and the documentation within, you must implement the following three class methods, marked by the comment *`Your code goes here`*:

```
// in CSRMatrix.cpp
CSRMatrix::CSRMatrix(const Rnmatrix &A);
Rnvector CSRMatrix::multiply(const Rnvector &x) const;

// in CGSolver.cpp
void CGSolver::solve(bool use_preconditioner);
```

The constructor builds the CSR representation of the dense $n \times n$ matrix `A` (with a_{ij} = `A[i][j]`): it must fill the members `values`, `col_idx` and `row_ptr`, storing only the entries of `A` different from zero, in the order described in Section 1.4. The number of rows `n` and the vector `row_ptr`, of size $n + 1$ with all elements equal to zero, are already initialized in the provided code. The method `multiply` *returns* the matrix-vector product $\mathbf{y} = A\mathbf{x}$, computed with Equation (3), i.e., without accessing the zero entries of A. The method `solve`

implements Algorithm 1: if its argument `use_preconditioner` is `false`, it runs plain CG; if it is `true`, it runs Jacobi-preconditioned CG. It does not return anything, but it must fill the following members of the class:

- `solution`: the final approximation $\mathbf{x}_k$;
- `residuals`: the residual norms $\|\mathbf{r}_0\|, \|\mathbf{r}_1\|, \ldots, \|\mathbf{r}_k\|$, in this order;
- `iterations`: the number k of iterations performed.

The first lines of `solve`, already provided, reset these members and set `solution` equal to the initial guess $\mathbf{x}_0$ (member `x0`). The tolerance ε and the maximum number of iterations $k_{\max}$ are stored in the members `tol` and `max_it`, respectively, and the preconditioner in the member `preconditioner`. The method `save_sol_to_file`, whose implementation is provided, saves these three members to a text file (see the format below): this is why `solve` must fill them exactly as described above.

Note: until the constructor of `CSRMatrix` is implemented, the CSR matrix is empty and the program stops with the message `Error: the diagonal entries of A must be positive!`, printed by the `JacobiPreconditioner`. We therefore suggest implementing the three methods in the order above.

Important: only the `CSRMatrix.cpp` and `CGSolver.cpp` files should be modified, and only inside the three methods listed above. Your implementations must be consistent with the method declarations and the class structure of the rest of the project. All other functions and methods defined in the code are available for use. The project compiles with strict warning flags (e.g., `-Werror`): use `std::size_t` for indices and avoid unused variables.

The `main.cpp` file runs both CG and PCG on two tridiagonal test systems of size $n = 50$, a well scaled one (Test 1) and a badly scaled one (Test 2), with initial guess $\mathbf{x}_0 = \mathbf{0}$, $\varepsilon = 10^{-7}$ and $k_{\max} = 500$. In order to compare your results with the provided ones, do not change these parameters. Executing the `main` will write the results to the four files `solution_1_CG.txt`, `solution_1_PCG.txt`, `solution_2_CG.txt` and `solution_2_PCG.txt`. Copies of these files, with the suffix `_copy`, are included in the zipped folder for comparison with the assignment solution. The files have the following structure (one solution component per line):

```
Method: CG
Iterations: 11
Solution:
0.344959
0.379836
[...]
Residual norms:
7.07107 1.1992 0.14487 0.0437148 [...] 1.17803e-06 3.38977e-07
```

Note: because of round-off errors, if you perform the operations in a different order (or use a different compiler), the last digits of the smallest residual norms

may differ slightly from the reference. The evaluation takes this into account, but the number of iterations, the solution and the residual norms must match the provided files.

Important: during the evaluation, we will test your code on the provided test systems *and* on additional ones that are not available to you, with a different SPD matrix (not necessarily tridiagonal) and a **non-zero** initial guess $\mathbf{x}_0$.

3 Delivery instructions

This assignment is not mandatory. However, if your implementation is correct, it will earn you an additional +1 point on the final grade.
Please follow these instructions for submission:

- Download the zipped folder `Assignment1_initial.zip` from WeBeep.
- Unzip the folder.
- Rename the **unzipped** folder as `YourCodicePersona` (e.g., `10699999`). **Important:** do this **before** re-zipping the project with your solution.
- Implement your code in the provided files.
- Test the code by executing the `main` and comparing the results from your files with the provided baseline.
- Zip the entire project folder again, making sure that the resulting file name is `YourCodicePersona.zip` (e.g., `10699999.zip`).
- Upload the file on WeBeep → Assignments → Assignment 1.

Attention: This assignment is individual; no group or team work is allowed. In cases of plagiarism, a penalty of 2 points (i.e., –2 on the total score) will be applied, and you will be excluded from future assignments. You are fully responsible for the code you submit. Using generative AI tools may produce code that closely resembles that of other students, potentially triggering plagiarism detection and leading to the penalties described above.
If you have questions, please post them on the WeBeep Assignments forum. **No feedback will be provided in the last 24 hours before the deadline.**
Code submission opens on Thursday, October 15th at 18:00 and closes on Wednesday, October 21st at 23:59 (Rome time).
If any updates are made to the provided source code, you will be informed through an announcement. Make sure to regularly check WeBeep during the submission period.